Building Multi-Agent System

Subagent Specs - System Prompt

What it is:

- Available agents (sub-agents/specialized agents) are hardcoded directly into the main agent's system instructions.
- The main agent sees agent names, descriptions, and capabilities as part of its base prompt.

When to use:

- Small, fixed set of agents (< 10)
- Agent registry rarely changes
- Want simplest implementation
- Single dispatch tool pattern

Business Scenario:

- Small startup with 3 agents: research, writer, reviewer
- Main agent needs to know which agents are available
- User: "Research AI trends and write a blog post"

```
main_agent = create_agent(
    model="...",
    tools=[task],
    system_prompt=(
        "You coordinate specialized sub-agents. "
        "Available agents:\n"
        "- research: Research and fact-finding\n"
        "- writer: Content creation and editing\n"
        "- reviewer: Code and document review\n"
    ),
)
```

How it works:

- **List agents in prompt** - Main agent sees available agents
- **Simple implementation** - No extra tools needed
- **Static list** - Must update prompt when agents change

Building Multi-Agent System

Subagent Specs - Enum Constraint

What it is:

- Available agents are defined as enum values in the dispatch tool's parameter schema, restricting agent selection to predefined options.

When to use:

- Small, fixed set of agents (< 10)
- Want type safety and explicit agent names
- Prefer schema-based validation over prompt-based guidance
- Single dispatch tool pattern

```python
from enum import Enum

class AgentName(str, Enum):
    RESEARCH = "research"
    WRITER = "writer"
    REVIEWER = "reviewer"

@tool
def task(agent_name: AgentName, description: str) -> str:
    """Launch an ephemeral subagent for a task."""
    # ...
```

Business Scenario:

- Fixed set of agents that rarely changes
- Want to prevent typos in agent names
- Ensure main agent only calls valid agents

How it works:

- **Enum class** - Define available agent names
- **agent_name: AgentName** - Type-safe parameter
- **Schema validation** - Model must use valid agent name

Building Multi-Agent System

Subagent Specs - Tool-based Discovery

What it is:

- A dedicated tool that the main agent calls at runtime to retrieve available agents and their capabilities.

When to use:

- Many agents (> 10) or growing registry
- Agent registry changes frequently or is dynamic
- Want to reduce prompt size and token usage
- Different teams manage different agents independently
- Single dispatch tool pattern

Business Scenario:

- Large enterprise with 50+ agents across departments
- Agents added/removed frequently by different teams
- User: "Find someone to help with tax compliance"
- Main agent searches registry for tax-related agents

```python
@tool
def list_agents(query: str = "") -> str:
    """List available subagents, optionally filtered by query."""
    agents = search_agent_registry(query)
    return format_agent_list(agents)

@tool
def task(agent_name: str, description: str) -> str:
    """Launch an ephemeral subagent for a task."""
    # ...

main_agent = create_agent(
    model="...",
    tools=[task, list_agents],
    system_prompt="Use list_agents to discover subagents, then use task to invoke them."
)
```

How it works:

- **list_agents tool** - Discover agents dynamically
- **search_agent_registry()** - Query available agents
- **Progressive disclosure** - Load agent info on-demand

Building Multi-Agent System

Subagent Inputs

What it is:
- Customize what context subagent receives
- Pull from agent's state for dynamic context

When to use:
- Subagent needs conversation history to understand context
- Need to pass metadata or state between agents
- Want to transform input before passing to subagent
- Subagent requires specific context not in query alone

```python
from langchain.agents import AgentState
from langchain.tools import tool, ToolRuntime

class CustomState(AgentState):
    order_id: str

@tool("shipping_agent", description="Handle shipping and delivery issues")
def call_shipping_agent(query: str, runtime: ToolRuntime[None, CustomState]):
    # Pass conversation history and order ID
    subagent_input = format_context(query, runtime.state["messages"])
    result = shipping_agent.invoke({
        "messages": subagent_input,
        "order_id": runtime.state["order_id"]
    })
    return result["messages"][-1].content
```

Business Scenario:
- Customer support: User has ongoing conversation about order issue
- Main agent calls shipping subagent
- Shipping subagent needs full conversation history to understand context
- Pass conversation history and order ID to shipping subagent

How it works:
- **ToolRuntime** - Access agent state
- **runtime.state["messages"]** - Get conversation history
- **format_context()** - Transform context for subagent
- **Pass state keys** - Share state between agents

Building Multi-Agent System

Subagent Outputs

What it is:

- Customize what main agent receives back
- Enrich response with additional state

When to use:

- Main agent needs more than just text response
- Subagent produces data that main agent will use for next steps
- Want to pass state from subagent back to main agent
- Need structured data in addition to conversational response

```python
from langgraph.types import Command
from langchain.tools import InjectedToolCallId

@tool("inventory_agent", description="Check product inventory")
def call_inventory_agent(
    query: str,
    tool_call_id: Annotated[str, InjectedToolCallId],
) -> Command:
    result = inventory_agent.invoke({"messages": [{"role": "user", "content": query}]})
    return Command(update={
        "warehouse_location": result["warehouse_location"],
        "stock_count": result["stock_count"],
        "messages": [
            ToolMessage(
                content=result["messages"][-1].content,
                tool_call_id=tool_call_id
            )
        ]
    })
```

Business Scenario:

- E-commerce: User asks to check inventory and place order
- Inventory subagent checks stock and returns availability + warehouse location
- Main agent needs both text response and warehouse location for order placement
- Subagent returns enriched data for main agent to use

How it works:

- **Command(update={...})** - Return enriched response
- **Pass additional state** - Share warehouse location and stock count
- **ToolMessage** - Properly formatted tool response
- **Main agent uses data** - Can place order with warehouse info

Building Multi-Agent System

Handoffs

What it is:

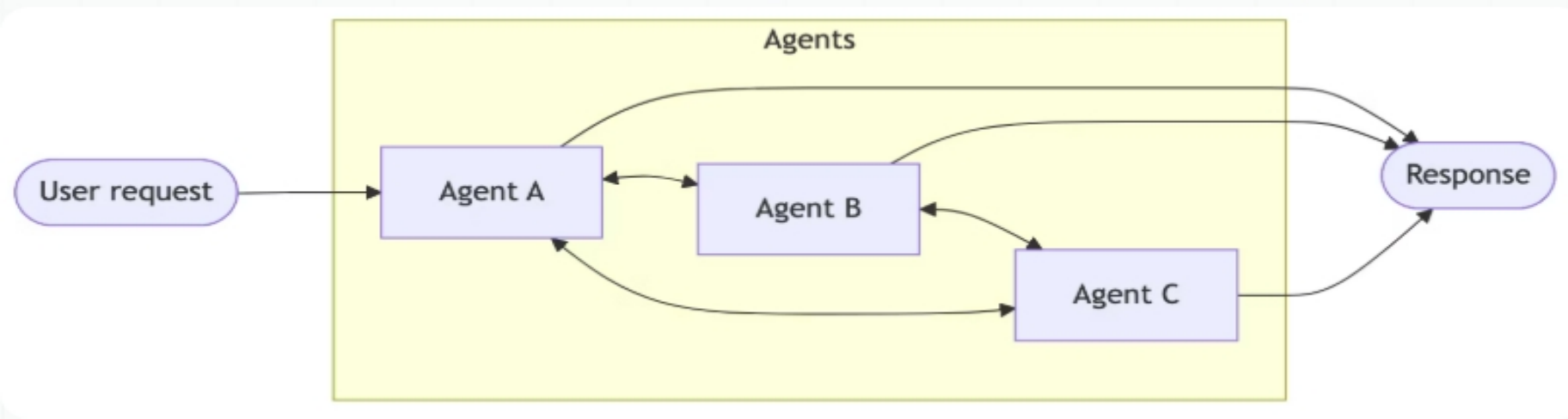
- Behavior changes dynamically based on state
- Tools update a state variable (e.g., current_step or active_agent)
- System reads variable to adjust behavior—apply different configuration or route to different agent
- Supports handoffs between distinct agents and dynamic configuration changes within single agent

When to use:

- Need to enforce sequential constraints (unlock capabilities only after preconditions met)
- Agent needs to converse directly with user across different states
- Building multi-stage conversational flows
- Customer support scenarios where you collect information in specific sequence

Two implementation approaches:

1. **Single agent with middleware** - One agent with dynamic configuration
2. **Multiple agent subgraphs** - Distinct agents as graph nodes



Building Multi-Agent System

Handoffs

What it is:

- Tool returns Command to update state
- Triggers transition to new step or agent
- ToolMessage completes request-response cycle

How it works:

- **@tool** - Define handoff tool
- **Command(update={...})** - Update state to trigger transition
- **ToolMessage** - Complete request-response cycle (required)
- **tool_call_id** - Match tool call with response
- **"current_step": "specialist"** - Update state variable to change behavior

```python
from langchain.tools import tool
from langchain.messages import ToolMessage
from langgraph.types import Command

@tool
def transfer_to_specialist(runtime) -> Command:
    """Transfer to the specialist agent."""
    return Command(
        update={
            "messages": [
                ToolMessage(
                    content="Transferred to specialist",
                    tool_call_id=runtime.tool_call_id
                )
            ],
            "current_step": "specialist"  # Triggers behavior change
        }
    )
```